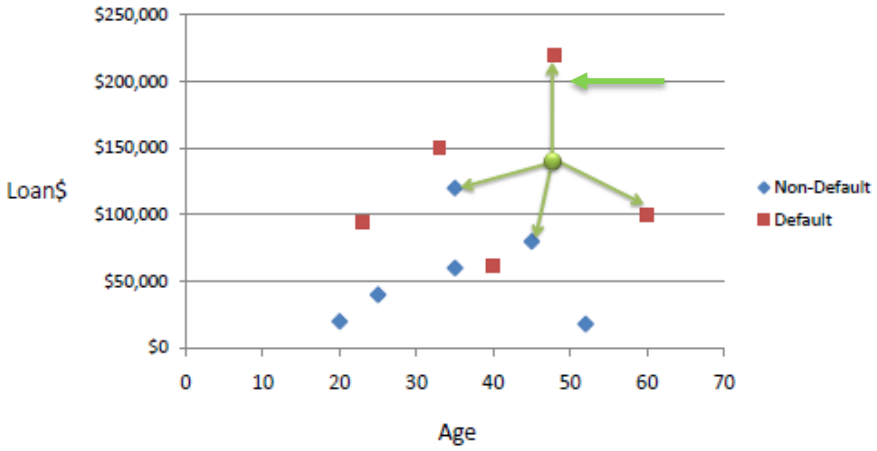
K Nearest Neighbor Classification

K Nearest Neighbor Classification

- K nearest neighbors is a simple algorithm that stores all available cases and classifies new cases based on a similarity measure (e.g., distance functions).
- A case is classified by a majority vote of its neighbors, with the case being assigned to the class most common amongst its K nearest neighbors measured by a distance function.
- No training phase — all computation happens at prediction time



K=5 case

What if k=1?
What if k=4?

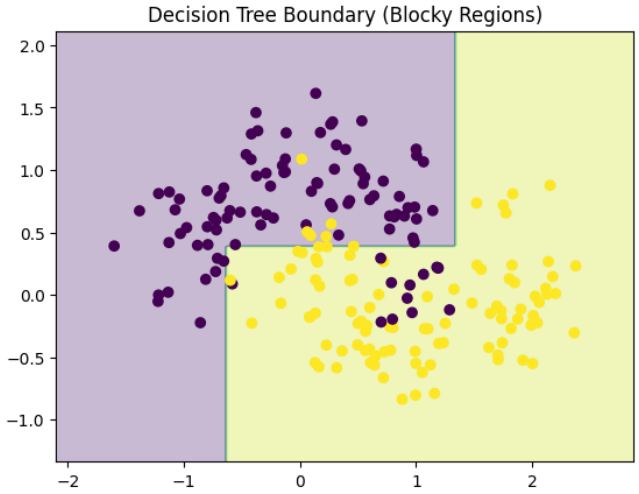
KNN Algorithm

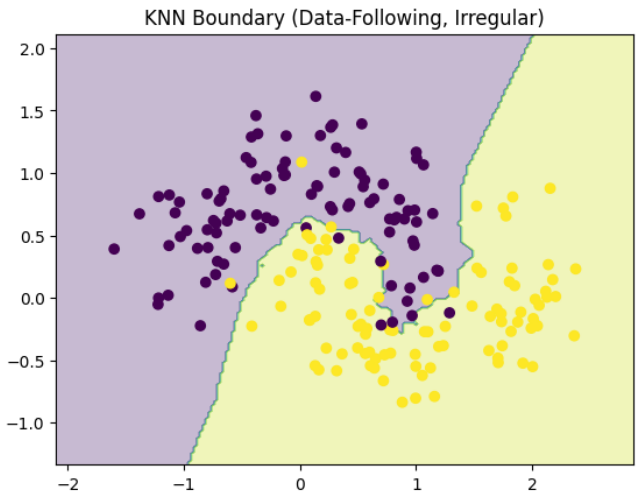
Input: training data, query point x, parameter k

1. Compute distance from x to all training points
2. Select k nearest neighbors
3. Classification → majority vote
   Regression → average value

KNN Challenges

- Why does such a simple method work so well?
  - KNN does **not assume a model** → it lets the data define the boundary
  - KNN can approximate *any* decision boundary given enough data.

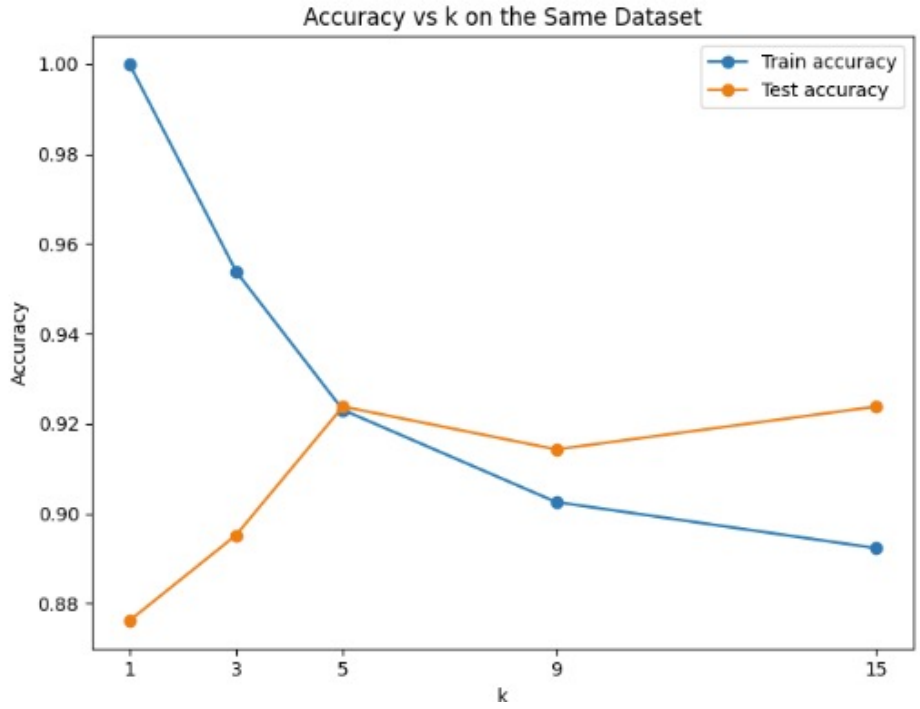




- Two key challenges in KNN:
  - How do we measure similarity? (already solved earlier)
  - How do we make it fast? (today's new part)

Key Design Choices

- Choosing K
  - k = 1 → very sensitive (overfitting)
  - large k → smoother boundary (underfitting)
- Effect of K
  - Demo:
    - Same dataset
    - K=1, 3, 5, 9, 15



Scaling Reminder

- Distance-based models require scaling
  - large-scale feature dominates

Strengths & Weaknesses

- **Pros:**
  - simple
  - no training
  - flexible decision boundary
- **Cons:**
  - slow prediction
  - sensitive to scaling
  - curse of dimensionality

Efficiency Problem → kd-tree & ball-tree

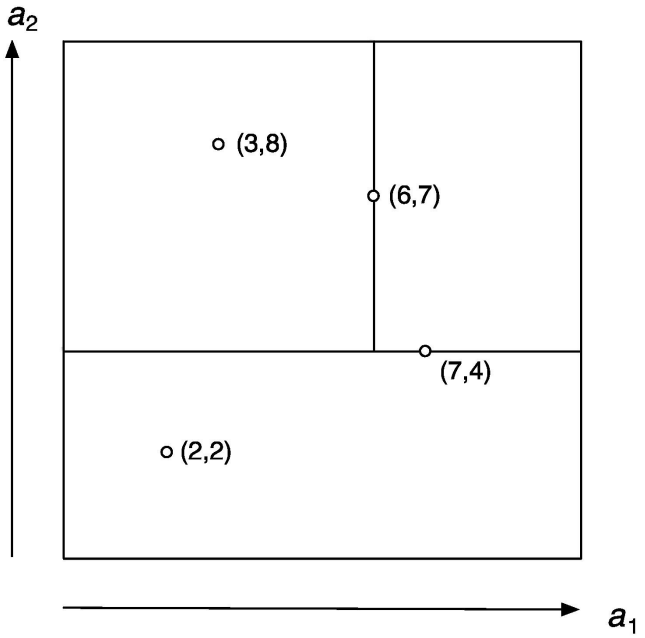
Finding nearest neighbors efficiently

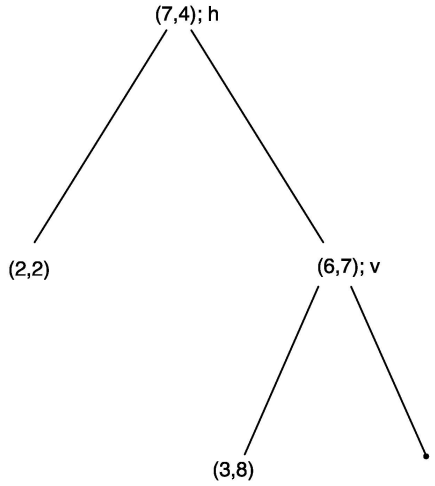
- Simplest way of finding nearest neighbor: linear scan of the data
  - Classification takes time proportional to the product of the number of instances in training and test sets
- Nearest-neighbor search can be done more efficiently using appropriate data structures
- Two methods that represent training data in a tree structure:

kD-trees and *ball trees*

(k-dimensional tree)

*k*D-tree example (2D case)



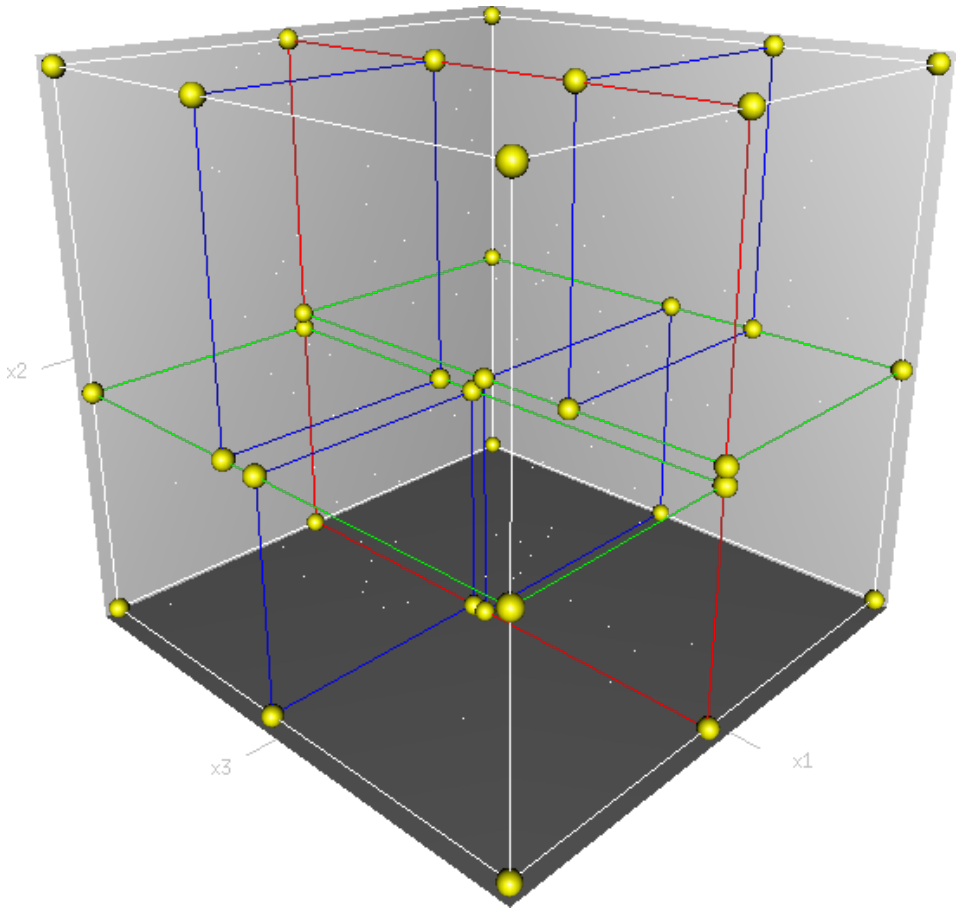


- Binary tree in which every node is a *k*-dimensional point.
- Every non-leaf node can be thought of as implicitly generating a splitting hyperplane that divides the space into two parts, known as half-spaces.

The hyperplane direction is chosen in the following way:
every node in the tree is associated with one of the k-dimensions,
with the hyperplane perpendicular to that dimension's axis.

*k*D-tree example (3D case)

- Binary tree in which every node is a *k*-dimensional point.
- Every non-leaf node can be thought of as implicitly generating a splitting hyperplane that divides the space into two parts, known as half-spaces.

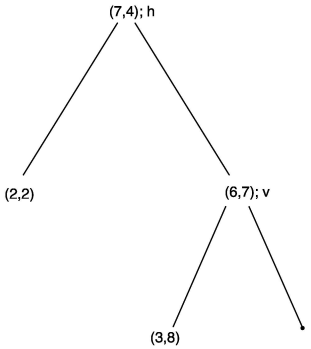


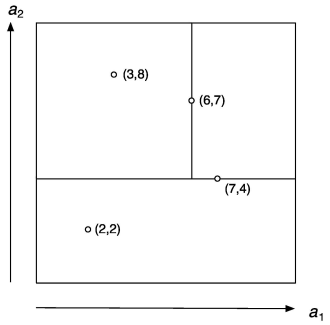
*k*D-tree example

```
 function kdtree (list of points pointList, int depth)
{
    // Select axis based on depth so that axis cycles through all valid values
    var int axis := depth mod k;

    // Sort point list and choose median as pivot element
    select median by axis from pointList;

    // Create node and construct subtree
    node.location := median;
    node.leftChild := kdtree(points in pointList before median, depth+1);
    node.rightChild := kdtree(points in pointList after median, depth+1);
    return node;
}
```





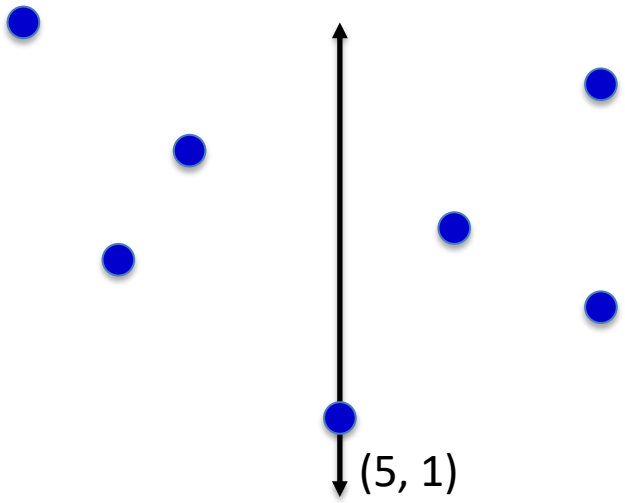
Building kD-trees

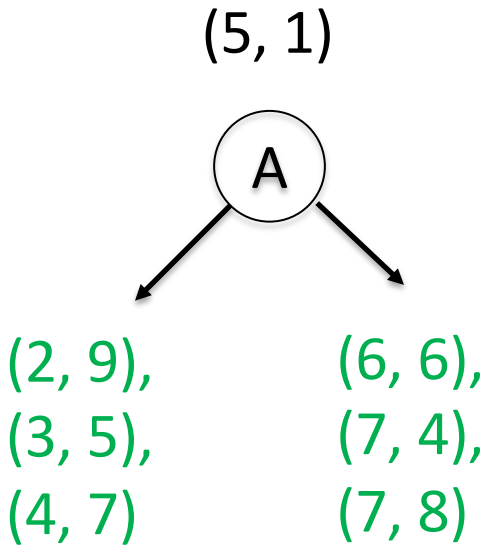
- Given the following 7 points, build the kd tree (k=2)

  (7, 8), (4, 7), (2, 9), (7, 4), (5, 1), (3, 5), (6, 6)
- Select an axis for splitting, lets say start with x-axis
- Find the median value of x-axis values

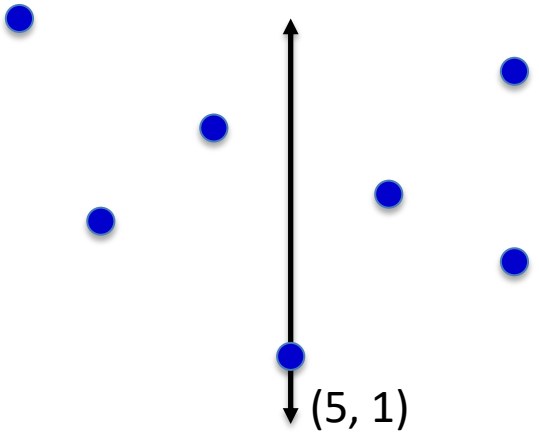
  sort the points along x-axis:

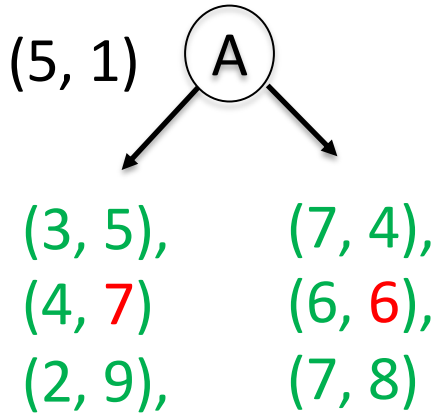
  (2, 9), (3, 5), (4, 7), (5, 1), (6, 6), (7, 4), (7, 8)
- split into two sub-trees:



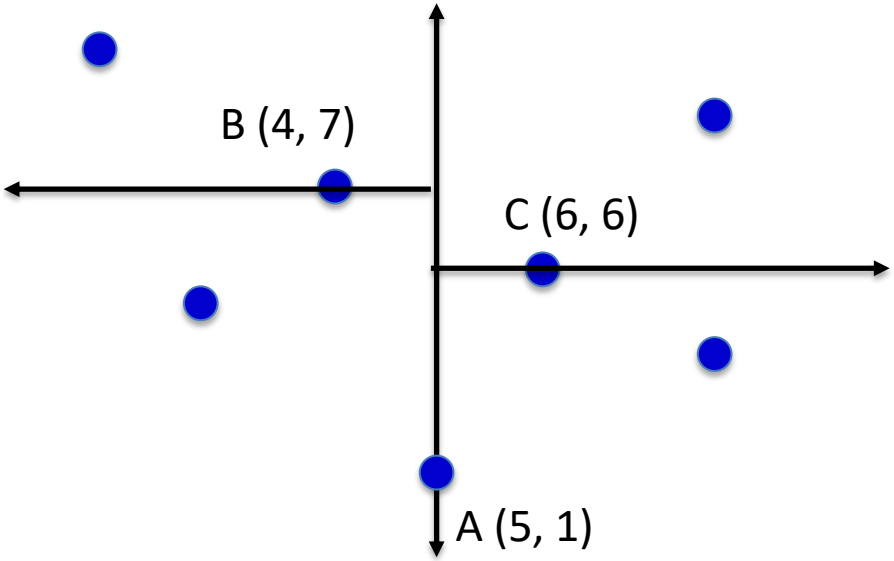


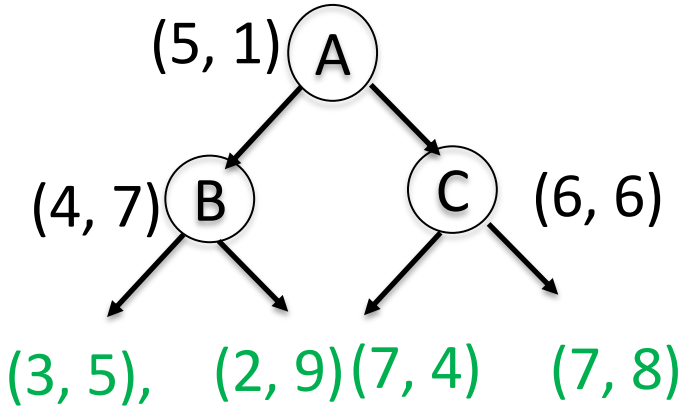
Building kD-trees





- In each sub-space, split the points along the next axis, ie., y-axis

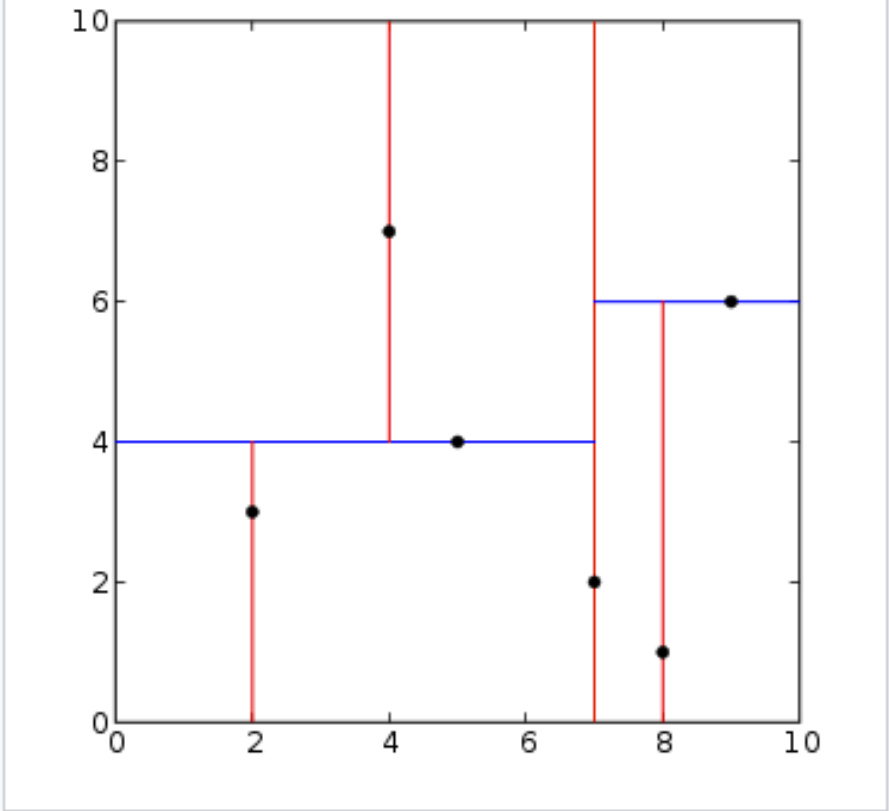




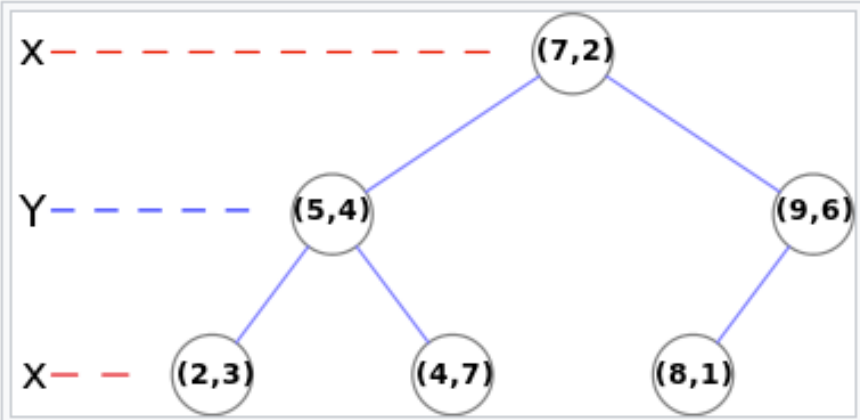
More on *k*D-trees

- Complexity depends on depth of tree, given by logarithm of number of nodes
- How to build a good tree? Need to find good split point and split direction
  - Split direction: direction with greatest variance
  - Split point: median value along that direction
- Using value closest to mean (rather than median) can be better if data is skewed
- Can apply this recursively

More Examples of Kd Tree



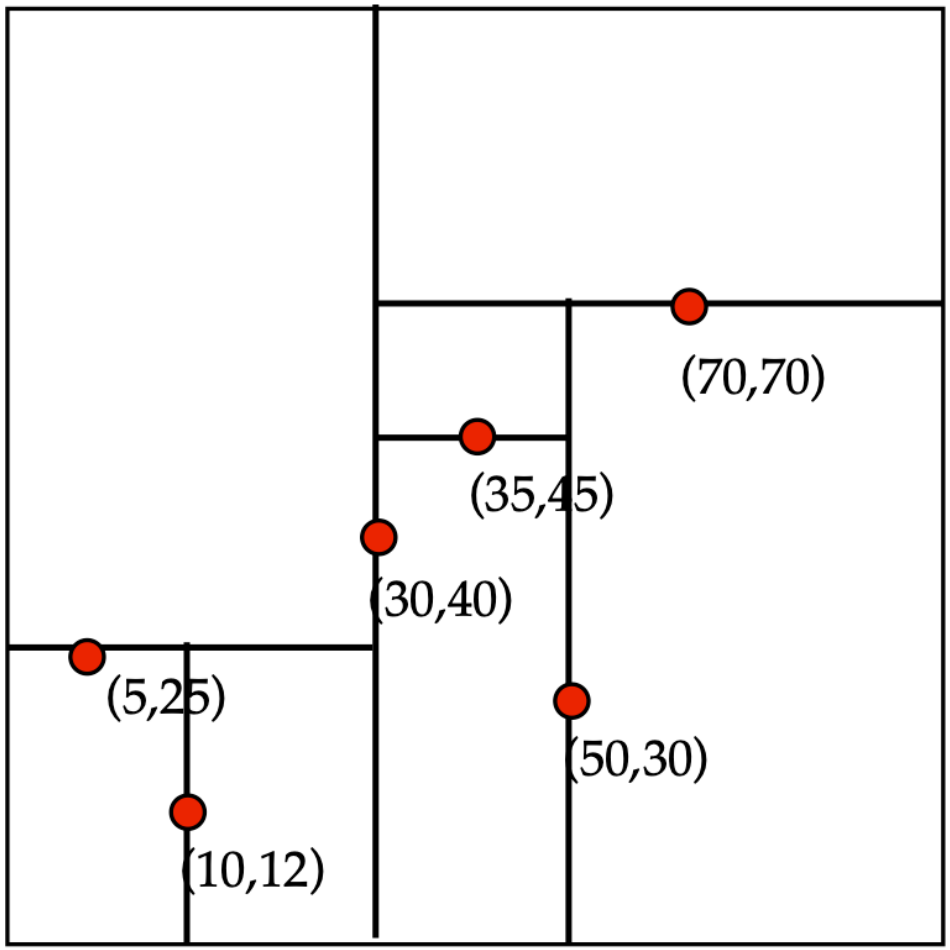
k-d tree decomposition for the point set `(2,3), (5,4), (9,6), (4,7), (8,1), (7,2)`.

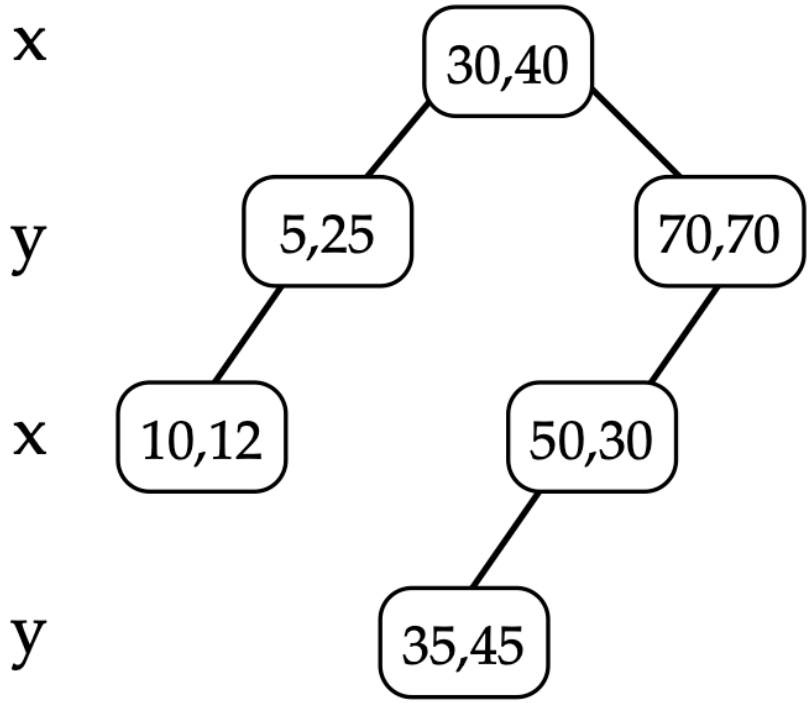


The resulting *k*-d tree.

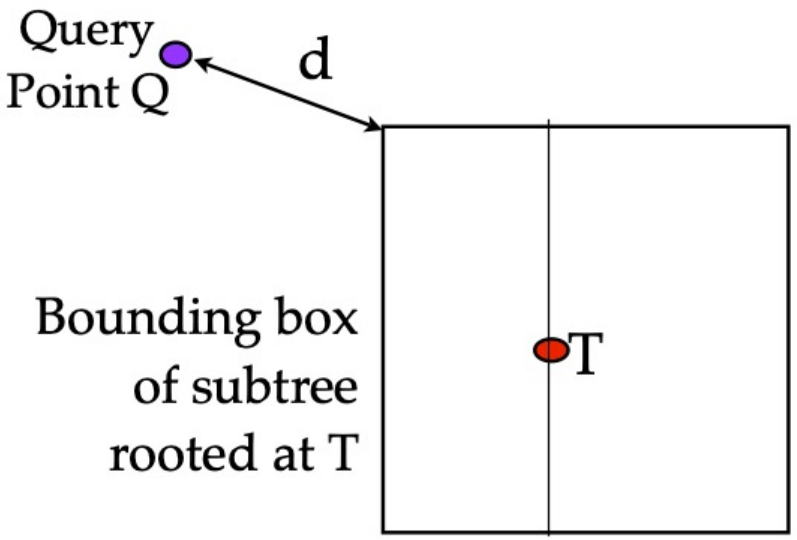
More Examples of Kd Trees

insert: (30,40), (5,25), (10,12), (70,70), (50,30), (35,45)





Using Kd Tree



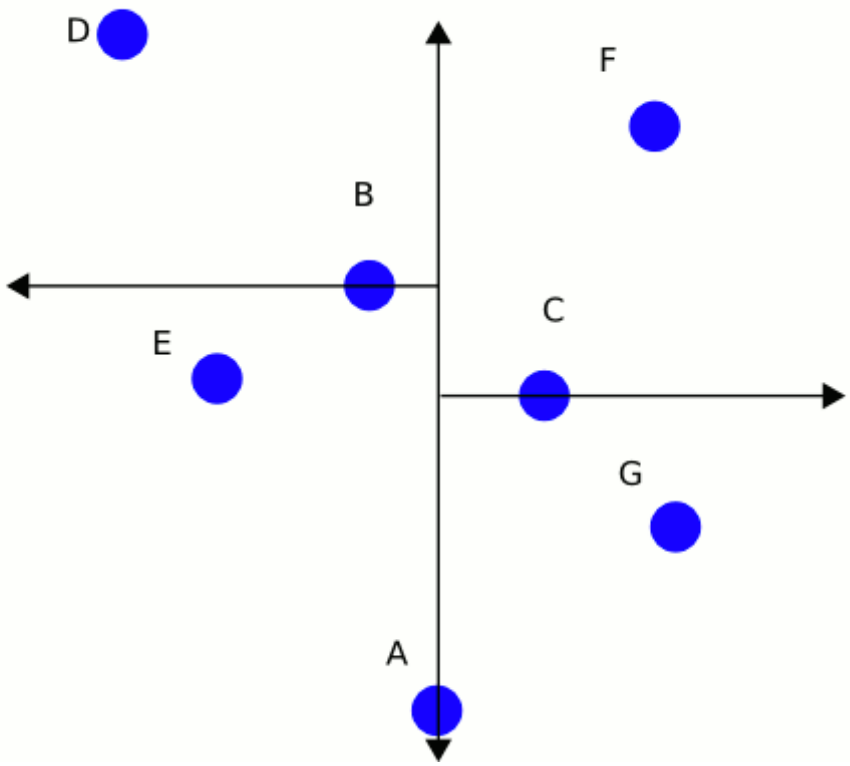
C -- current best point

If d > dist(C, Q), then no point in BB(T) can be closer to Q than C. Hence, no reason to search subtree rooted at T.

Update the best point so far, if T is better:
if dist(C, Q) > dist(T.data, Q), C := T.data

Recurse, but start with the subtree "closer" to Q:
First search the subtree that would contain Q if we were inserting Q below T.

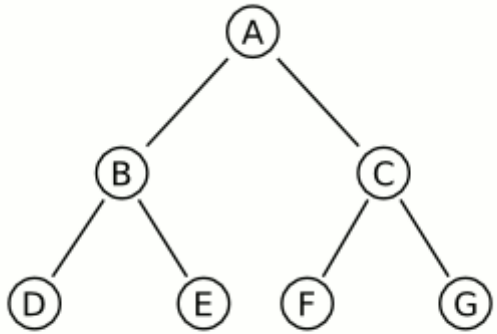
Using kD-trees



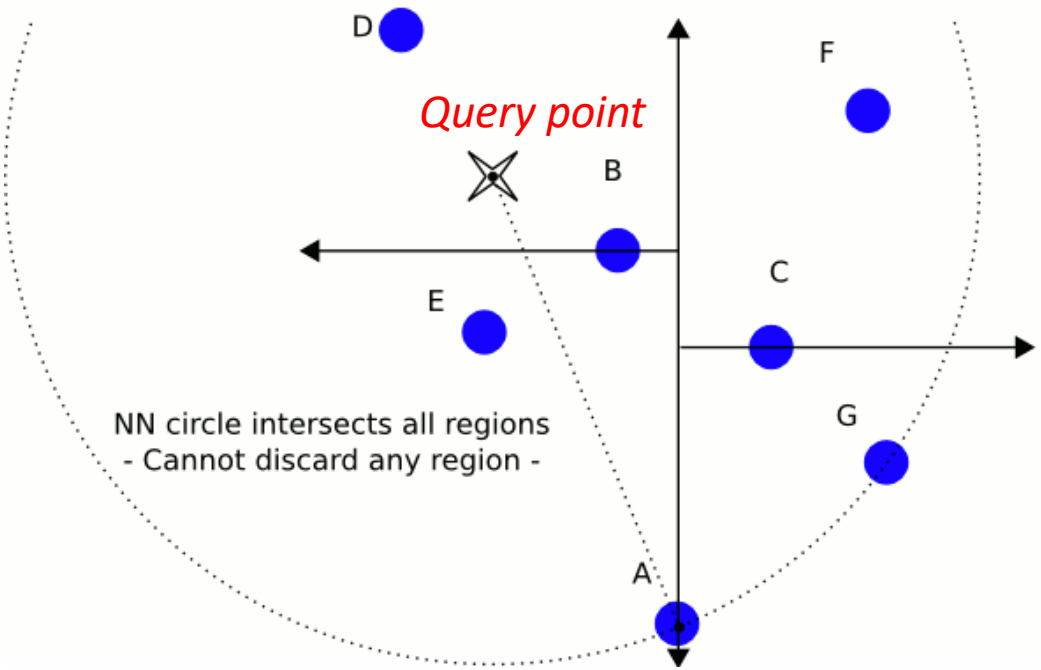
X-Splitting planes

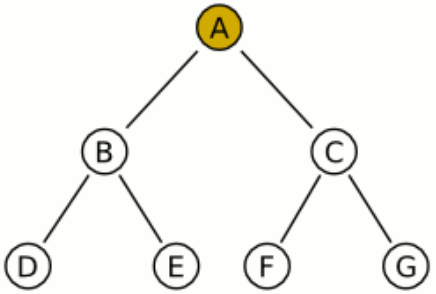
Y-Splitting planes

X-Splitting planes
not needed for leaf



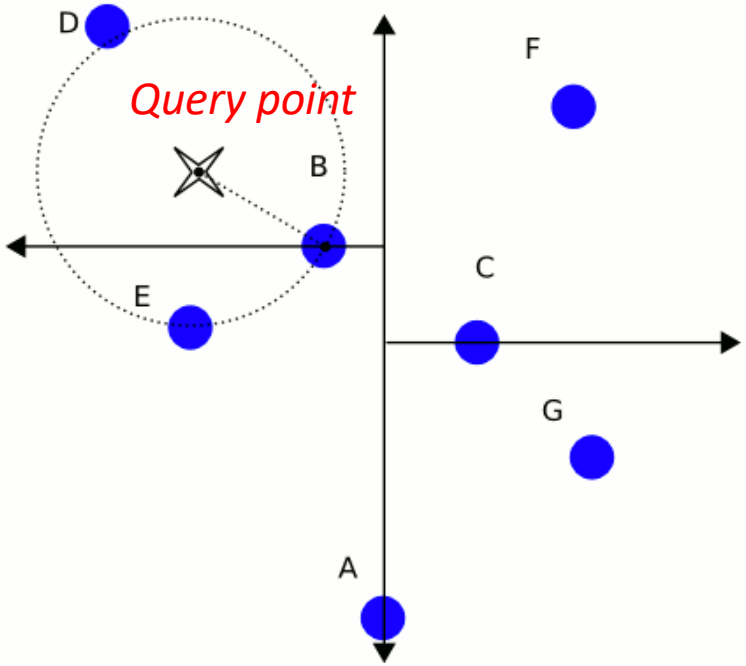
Using kD-trees

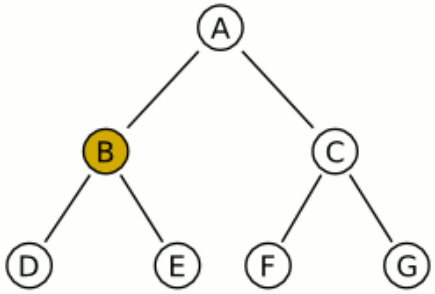




Start at A, then proceed in depth-first search (maintain a stack of parent-nodes if using a singly-linked tree). Set best estimate to A's distance Then examine left child node

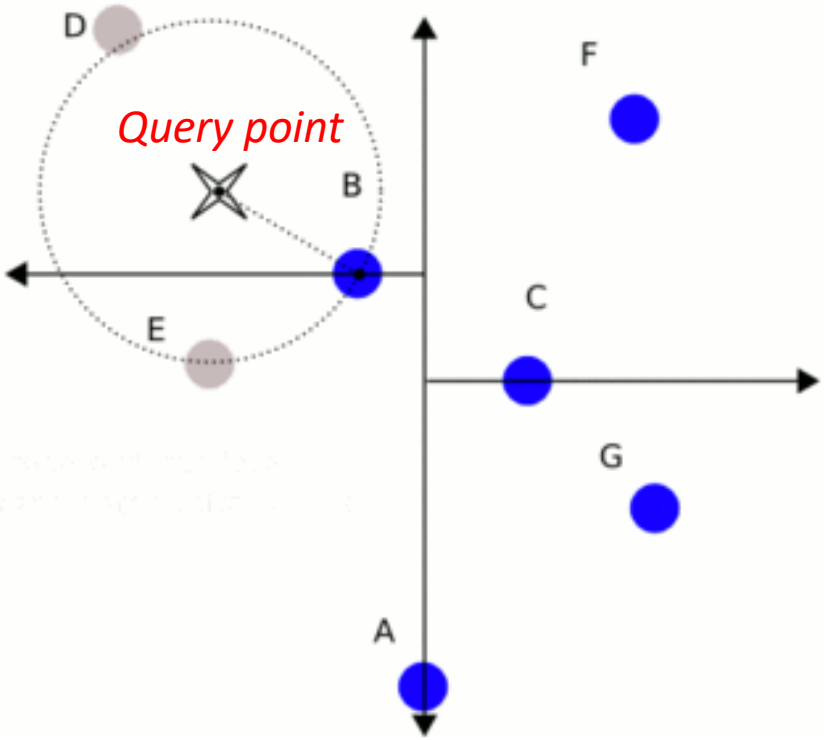
Using kD-trees

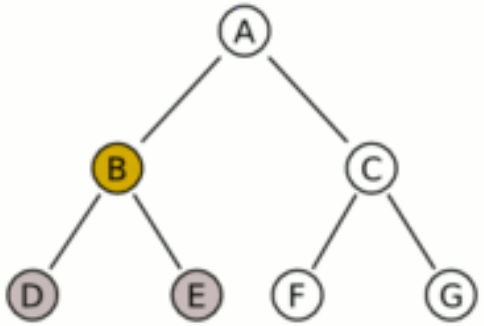




Calculate B's distance and
compare against best estimate
- It is smaller distance, so update
best estimate. Examine children (left then right)

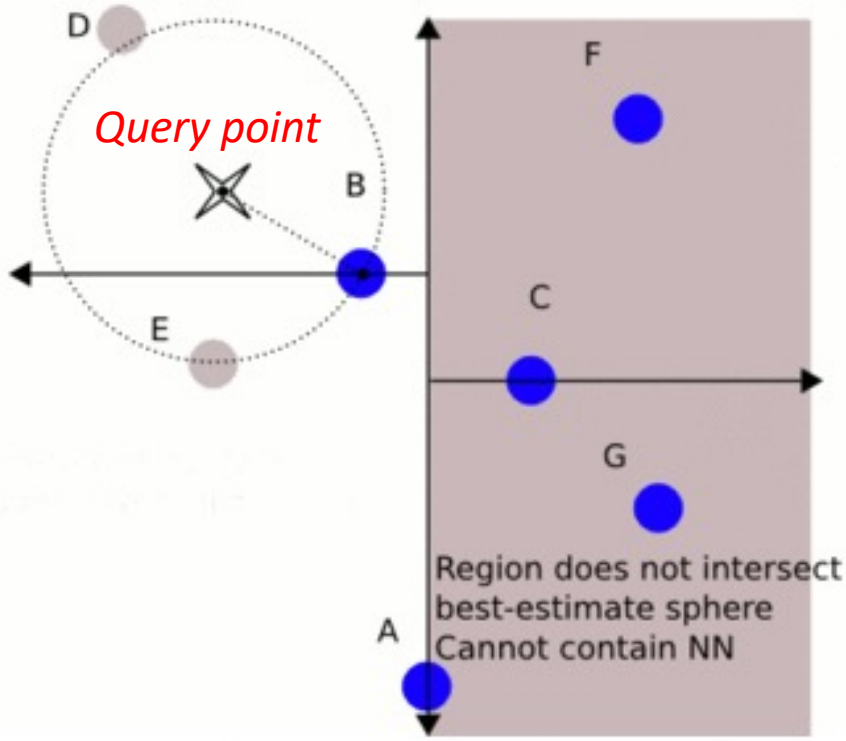
Using kD-trees

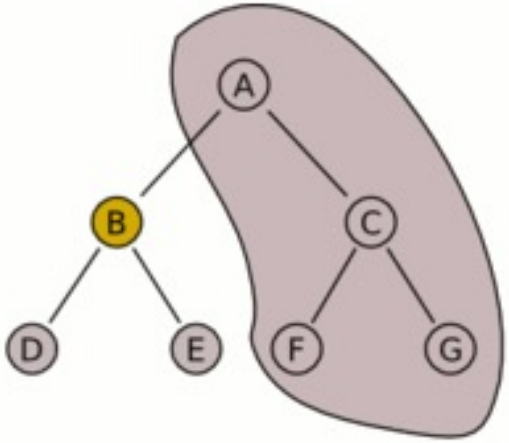




D & E Discarded as B
(already visited) is closer.
B is the best estimate for B's sub-branch
Proceed back to parent node

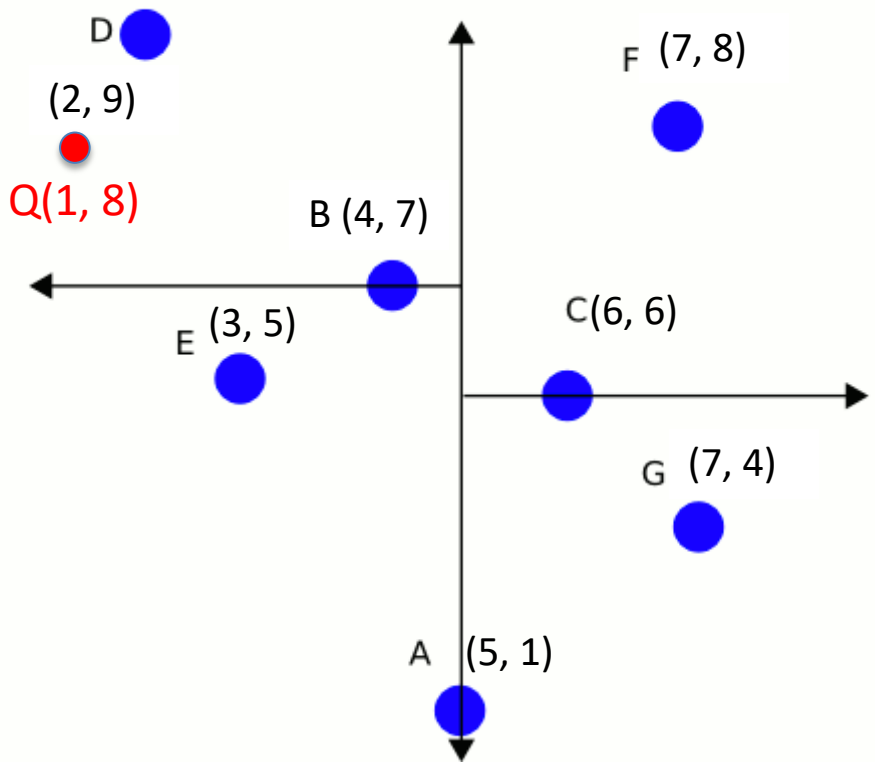
Using kD-trees





A's children have all been searched,
B is the best estimate for entire tree

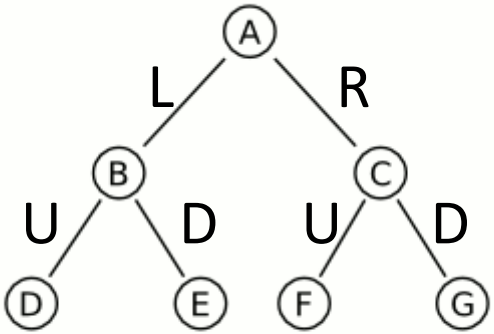
Using KD Tree Example



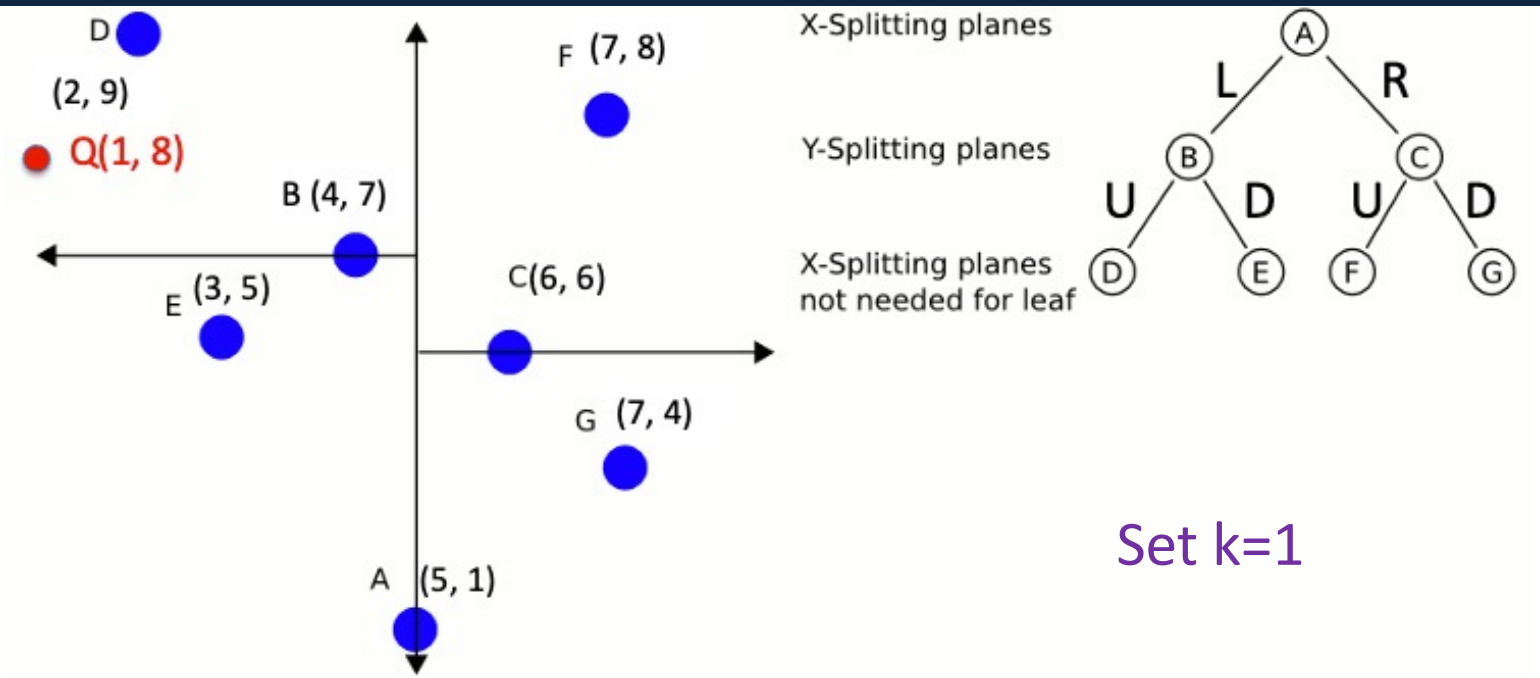
X-Splitting planes

Y-Splitting planes

X-Splitting planes not needed for leaf



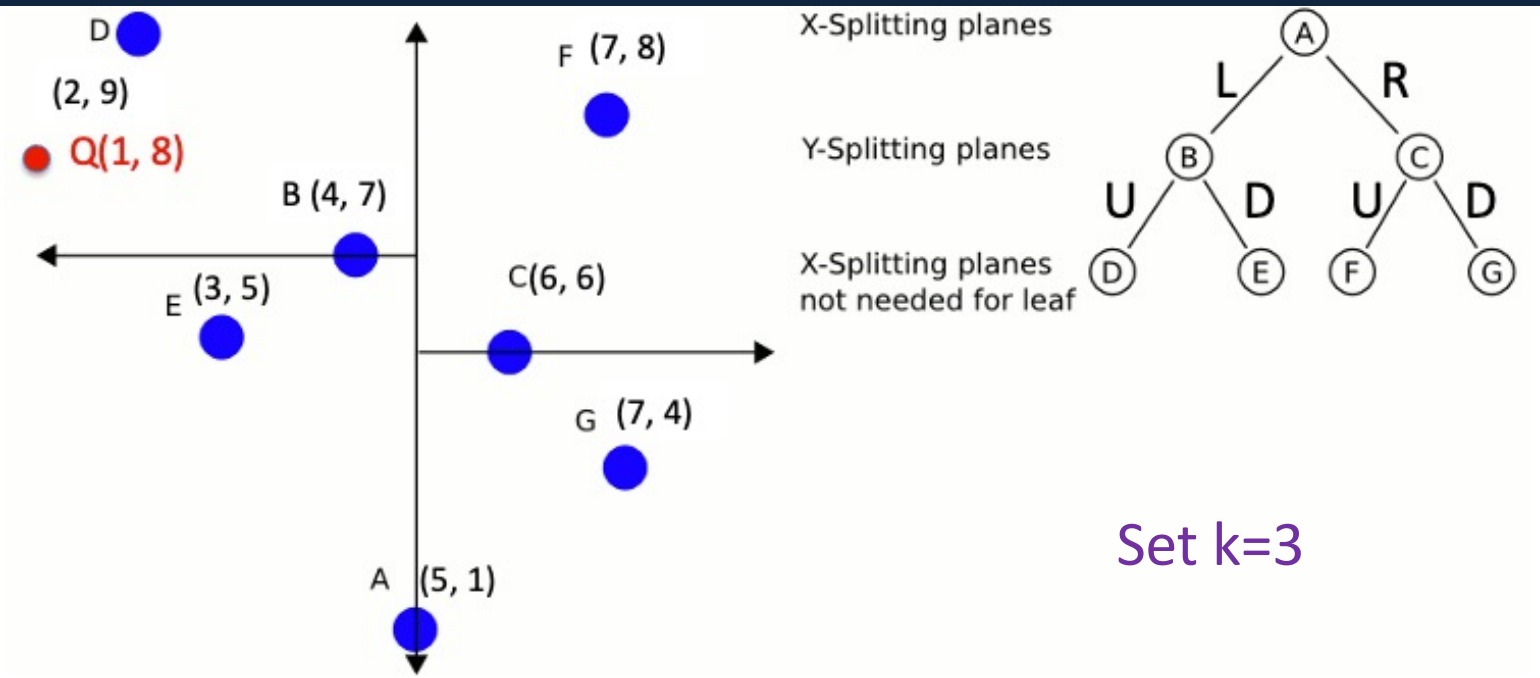
Using kD-trees



Set k=1

1. distance (Q, A), best estimate = 8.02 { }
2. Left, distance (Q, B)=3.16, update best estimate = 3.16 { }
3. Up, leaf node, distance (Q, D)= 1.414 , add D to top K neighbor queue {D(1.4)}
4. Backtrack, is there need to explore the lower half of the left tree? Yes, distance from x to split line is 1, less than current best 1.414 ...
5. Distance (Q, E) = 3.6, do not add to queue {D(1.4)}
6. Backtrack to B and then A, B and A's distance larger than D, {D(1.4)}
7. No need to explore the right side of A because the best distance is 4 > 1.4

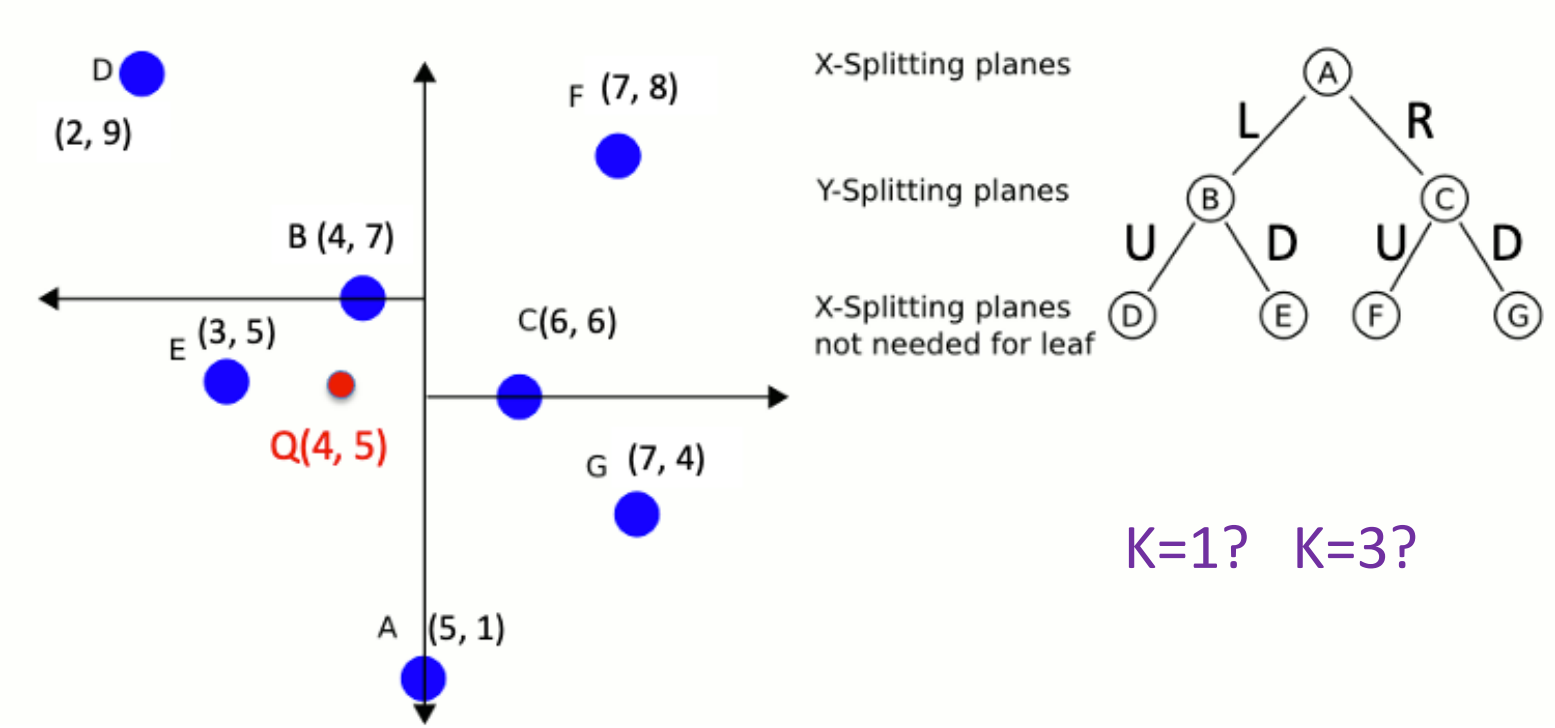
Using kD-trees



Set k=3

1. distance (Q, A), best estimate = 8.02 {}
2. Left, distance (Q, B)=3.16, update best estimate = 3.16 { }
3. Up, leaf node, distance (Q, D)= 1.414 , add D to top K neighbor queue {D(1.4)}
4. Backtrack, is there need to explore the lower half of the left tree? Yes, distance from x to split line is 1, less than current best 1.414 ...
5. Distance (Q, E) = 3.6, add to queue {E(3.6), D(1.4)}
6. Backtrack, add B to queue {E(3.6), B(3.16),D(1.4)}
7. No need to explore the right side of A because the best distance is 4 > 3.6

Practice Question



K=1? K=3?

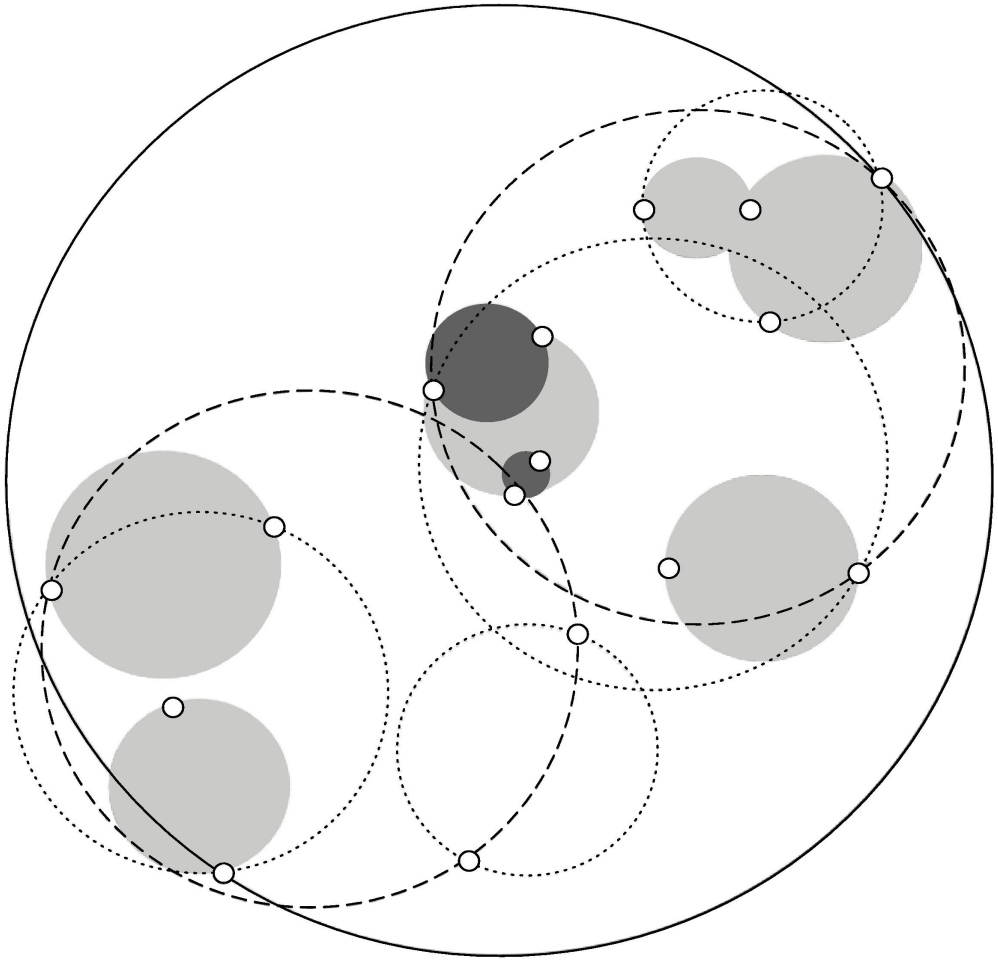
Building trees incrementally

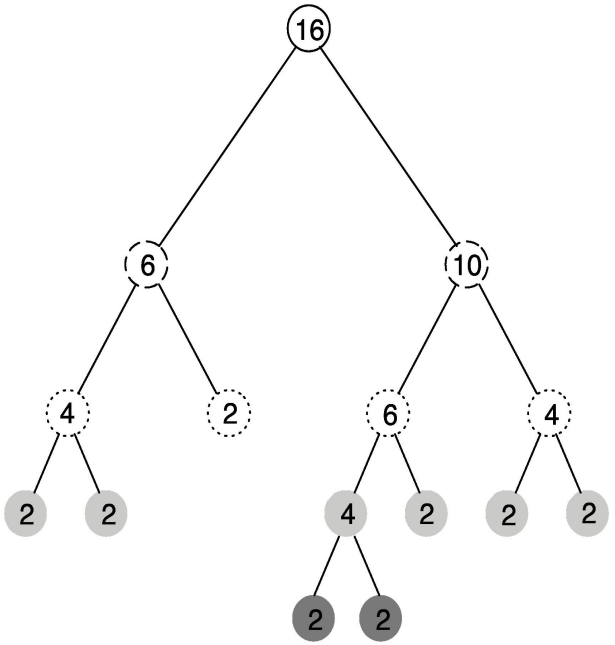
- Big advantage of instance-based learning: classifier can be updated incrementally
  - Just add new training instance!
- Can we do the same with *k*D-trees?
- Heuristic strategy:
  - Find leaf node containing new instance
  - Place instance into leaf if leaf is empty
  - Otherwise, split leaf according to the longest dimension (to preserve squareness)
- Tree should be re-built occasionally (i.e. if depth grows to twice the optimum depth)

Ball trees

- Observation: no need to make sure that regions don't overlap
- Can use balls (hyperspheres) instead of hyperrectangles
  - A *ball tree* organizes the data into a tree of *k*-dimensional hyperspheres
  - Normally allows for a better fit to the data and thus more efficient search

Ball tree example





Building ball trees

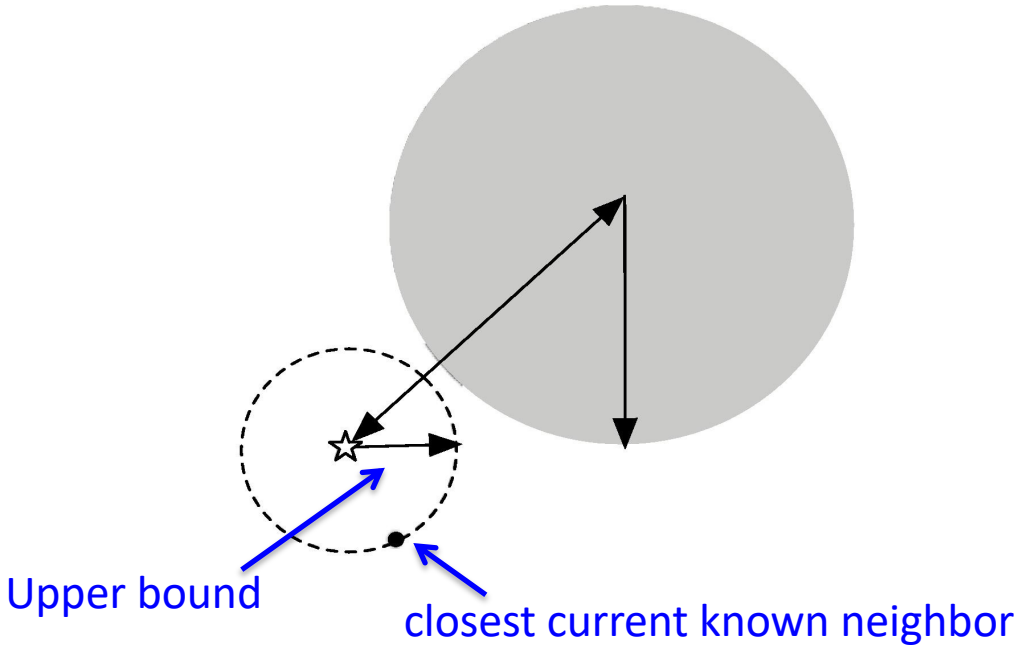
- Ball trees are built top down (like *k*D-trees)
- Basic problem: splitting a ball into two

```
function construct_balltree is
    input:
        D, an array of data points
    output:
        B, the root of a constructed ball tree
    if a single point remains then
        create a leaf B containing the single point in D
        return B
    else
        let c be the dimension of greatest spread
        let L,R be the sets of points lying to the left and right of the median along dimension c
        create B with two children:
            B.pivot = c
            B.child1 = construct_balltree(L),
            B.child2 = construct_balltree(R)
        return B
    end if
end function
```

Using ball trees

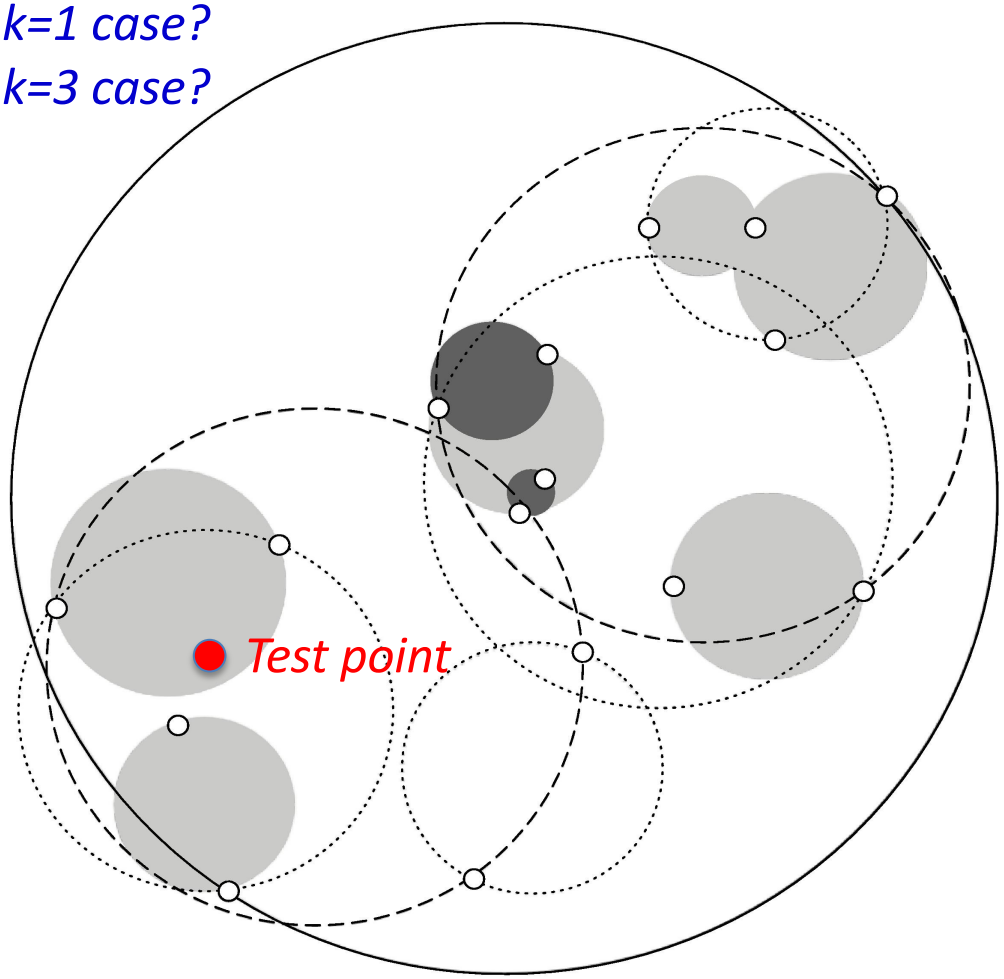
- Nearest-neighbor search is done using the same backtracking strategy as in *k*D-trees
- Ball can be ruled out from consideration if: distance from target to ball's center exceeds ball's radius plus current upper bound.

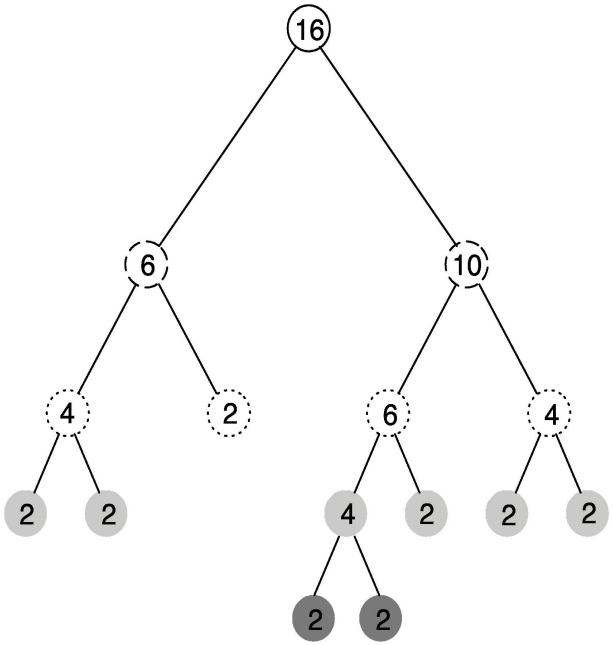
(two circles no overlap)



Ball tree example

k=1 case?
k=3 case?

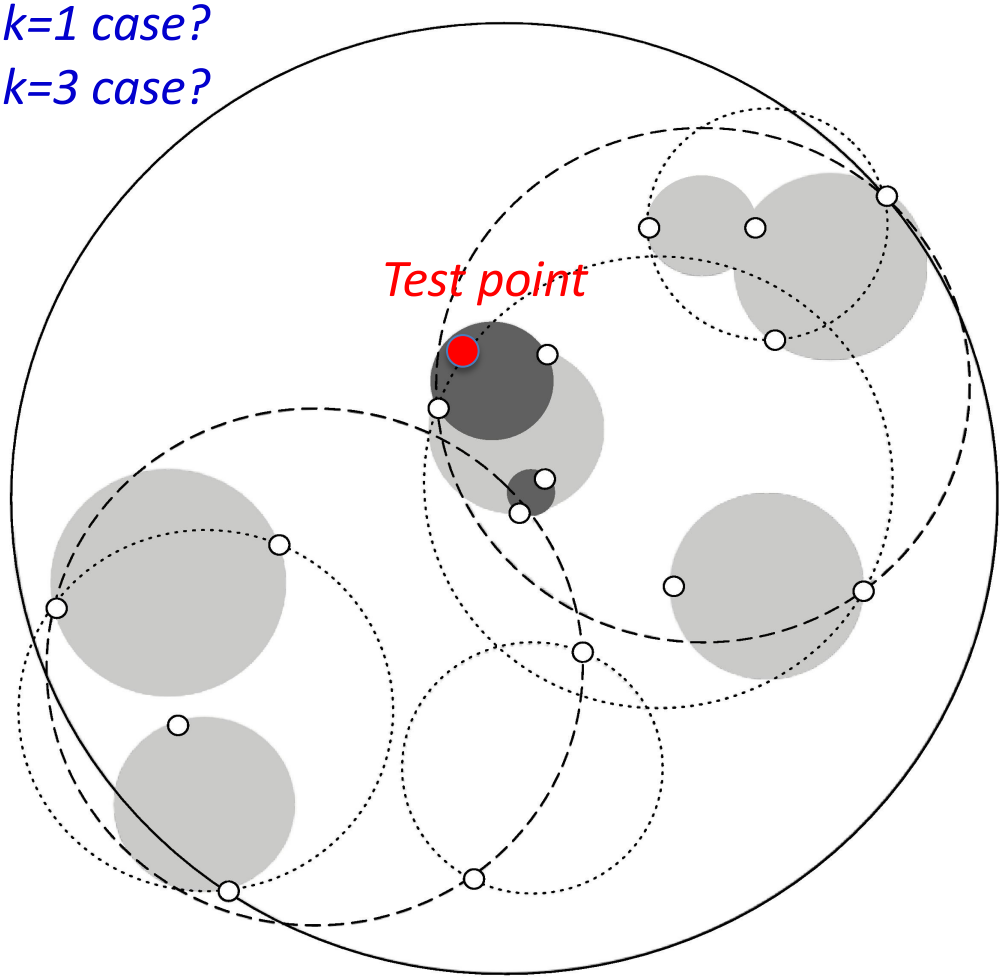


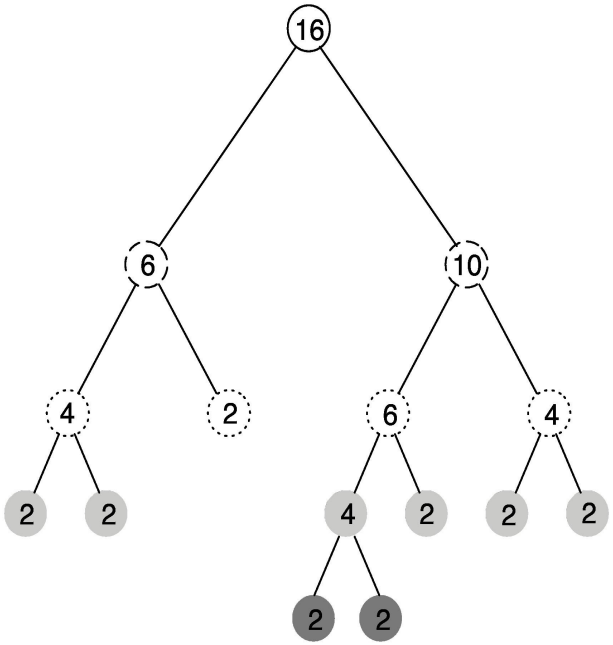


Ball can be ruled out from consideration if: distance from target to ball's center exceeds ball's radius plus current upper bound

Ball tree example

k=1 case?
k=3 case?





Ball can be ruled out from consideration if: distance from target to ball's center exceeds ball's radius plus current upper bound

Nearest Neighbor with Ball Tree

- At each node *B*, it may perform one of three operations, before finally returning an updated version of the priority queue:
  - If the distance from the test point *t* to the current node *B* is greater than radius + the furthest point in *Q,* ignore *B* and return *Q*.
  - If *B* is a leaf node, scan through every point enumerated in *B* and update the nearest-neighbor queue appropriately. Return the updated queue.
  - If *B* is an internal node, call the algorithm recursively on *B'*s two children, searching the child whose center is closer to *t* first. Return the queue after each of these calls has updated it in turn.